

Architecture Diagram

Protokol Manajemen Kunci & Komunikasi

Bacarita Security Design

Daftar Isi

1. Remediasi yang Diimplementasi	2
2. Arsitektur Komunikasi Aman	2
2.1. Diagram Alur Tingkat Tinggi	2
2.2. Komponen Keamanan per Lapisan	4
3. Protokol Manajemen Kunci	4
3.1. Inventaris Kunci Kriptografis	4
3.2. Siklus Hidup Token JWT	5
3.3. Diagram Verifikasi Token	5
4. Protokol Komunikasi	6
4.1. Spesifikasi Kanal	6
4.2. Header Keamanan HTTP	6
5. Perubahan Struktur Database	7

1. Remediasi yang Diimplementasi

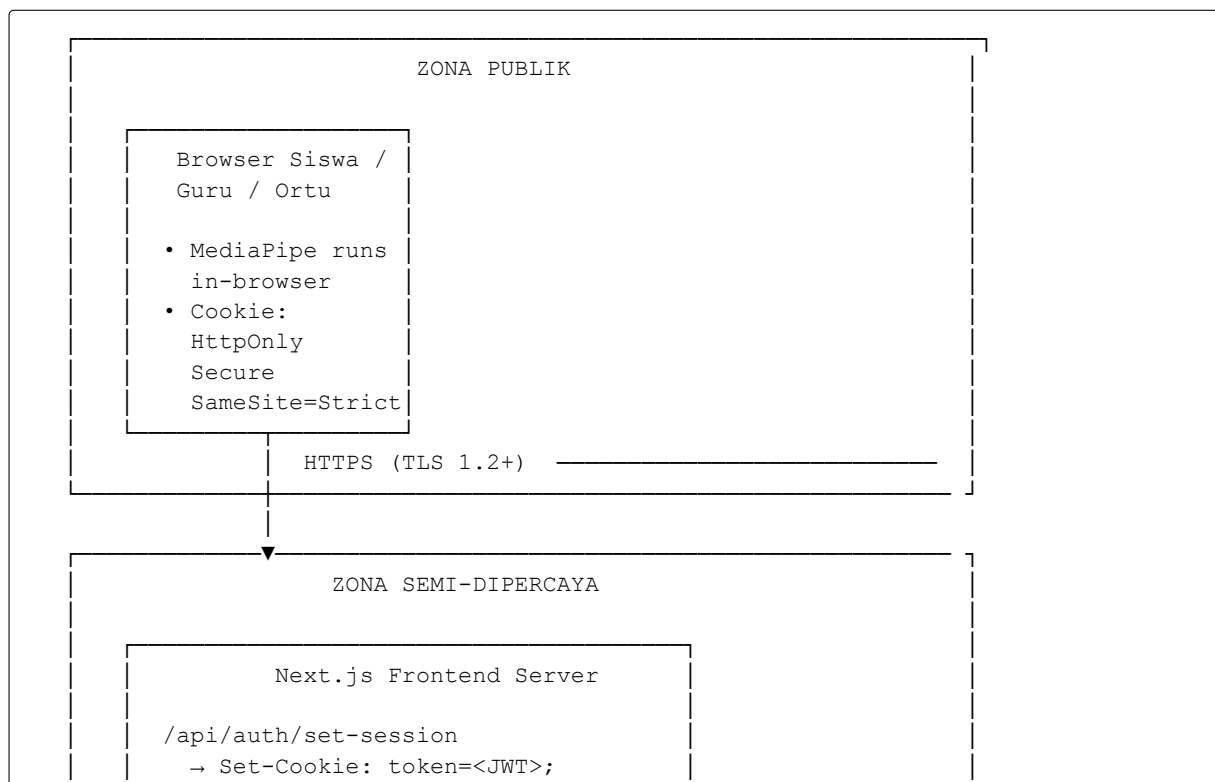
Dari hasil vulnerability assessment, dipilih lima remediasi dengan rasio **dampak tinggi terhadap effort rendah-sedang** yang langsung memperkuat protokol autentikasi dan manajemen kunci:

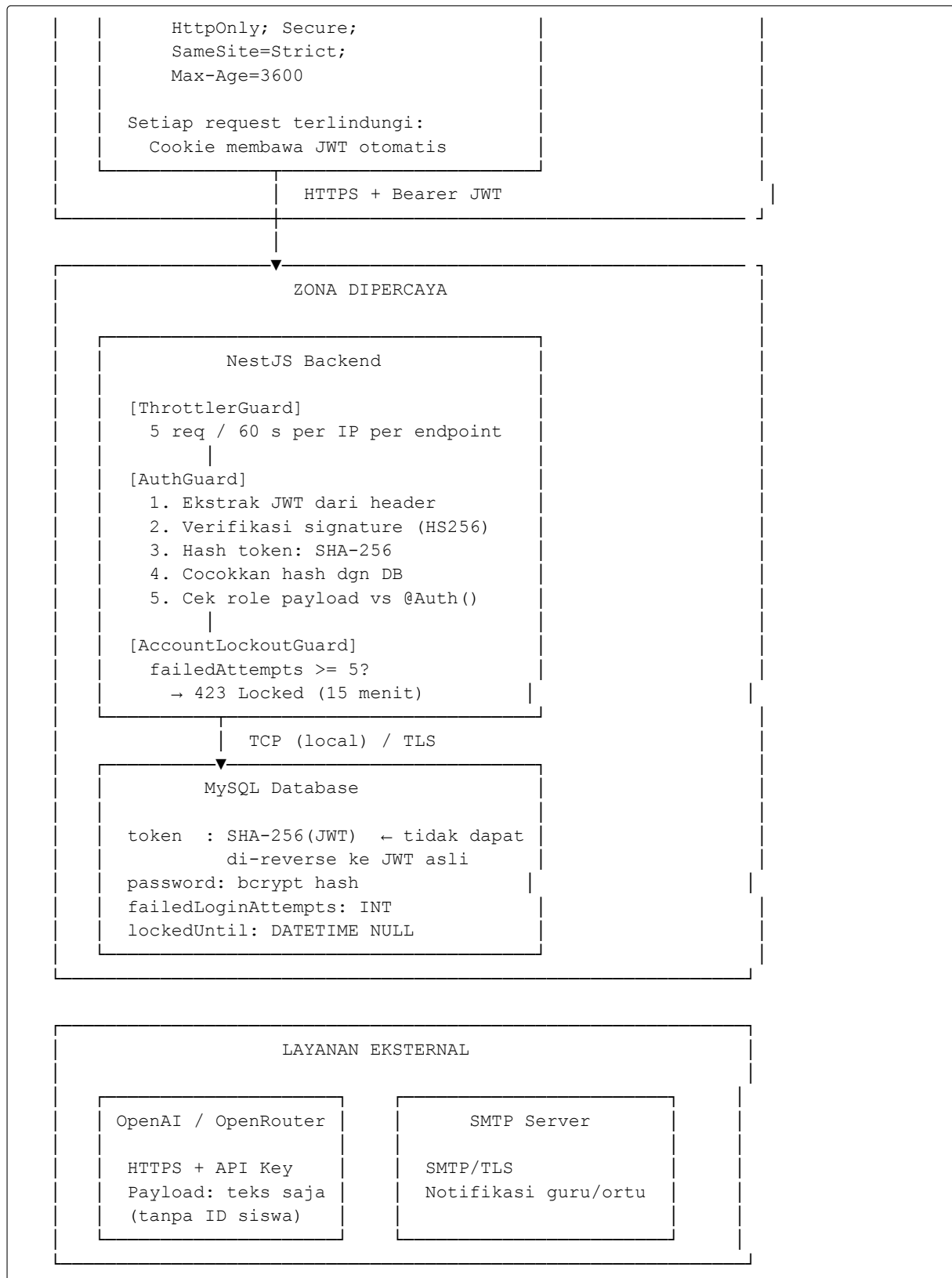
ID	Risiko Awal	Remediasi	Prioritas	Effort
AUTH-05	Kritis	Migrasi token ke HttpOnly + Secure + SameSite=Strict cookie via Next.js server	P1	Sedang
AUTH-01/02	Kritis	Rate limiting @nestjs/throttler — 5 req/60 dtk per IP pada semua endpoint login	P1	Rendah
AUTH-03	Kritis	Simpan hash SHA-256 token di database; bukan token JWT asli	P2	Sedang
USER-03	Tinggi	Account lockout: kunci akun 15 menit setelah 5 kali login gagal	P1	Sedang
VA-08	Sedang	Validasi kompleksitas password minimal 8 karakter di semua DTO	P3	Rendah

Tabel 1: Daftar remediasi yang diimplementasi

2. Arsitektur Komunikasi Aman

2.1. Diagram Alur Tingkat Tinggi





2.2. Komponen Keamanan per Lapisan

Lapisan	Kontrol Keamanan	Implementasi
Browser	Token disimpan di HttpOnly cookie; eye-tracking berjalan lokal	Set-Cookie: HttpOnly; Secure; SameSite=Strict
Next.js	Set cookie server-side; tidak ada token di JavaScript	/api/auth/set-session route
NestJS — Throttle	Rate limiting per IP per endpoint	ThrottlerModule, ThrottlerGuard
NestJS — Auth	Verifikasi JWT + cocokkan hash token ke DB	AuthGuard, jwtService.verify()
NestJS — RBAC	Periksa peran dari JWT payload vs dekorator	@Auth(AuthRole.*), Reflector
NestJS — Lockout	Blokir akun setelah 5 gagal; buka otomatis 15 menit	failedLoginAttempts, lockedUntil
MySQL	Token tersimpan sebagai SHA-256 hash; password sebagai bcrypt	token TEXT, password VARCHAR(255)
LLM Eksternal	Hanya teks cerita yang dikirim; tanpa konteks pengguna	Audit payload di open-router.service.ts

Tabel 2: Kontrol keamanan per lapisan

3. Protokol Manajemen Kunci

3.1. Inventaris Kunci Kriptografis

Kunci	Algoritma	Penyimpanan	Kebijakan
JWT Secret	HS256 (HMAC-SHA256)	.env → env var runtime	Min. 256-bit acak; rotasi jika terjadi kebocoran
Token Hash	SHA-256 (satu arah)	Kolom token di DB	Dibuat saat login; dihapus saat logout
Password Hash	bcrypt (cost factor ≥ 12)	Kolom password di DB	Hash saat registrasi; tidak pernah disimpan plain
API Key LLM	Bearer Token	.env → env var runtime	Rotasi berkala; tidak pernah dikirim ke frontend
SMTP Password	Autentikasi SMTP	.env → env var runtime	Akun email terpisah khusus sistem

Tabel 3: Inventaris dan kebijakan kunci kriptografis

3.2. Siklus Hidup Token JWT

LOGIN REQUEST

1. Klien kirim { username, password } via HTTPS POST /auth/{role}/login
2. Backend verifikasi password dengan bcrypt.compare()
3. Cek failedLoginAttempts dan lockedUntil
 - Jika akun terkunci → 423 Locked
 - Jika password salah → increment failedLoginAttempts
Jika >= 5 → set lockedUntil = now + 15 min
 - Jika benar → reset failedLoginAttempts = 0
4. Buat JWT: jwtService.sign({ id, email, username, role }, { expiresIn: '1h' })
5. Hitung tokenHash = SHA-256(JWT)
6. Simpan tokenHash ke kolom `token` di DB
7. Kembalikan { token: JWT } ke frontend
8. Frontend POST /api/auth/set-session → Next.js set HttpOnly cookie

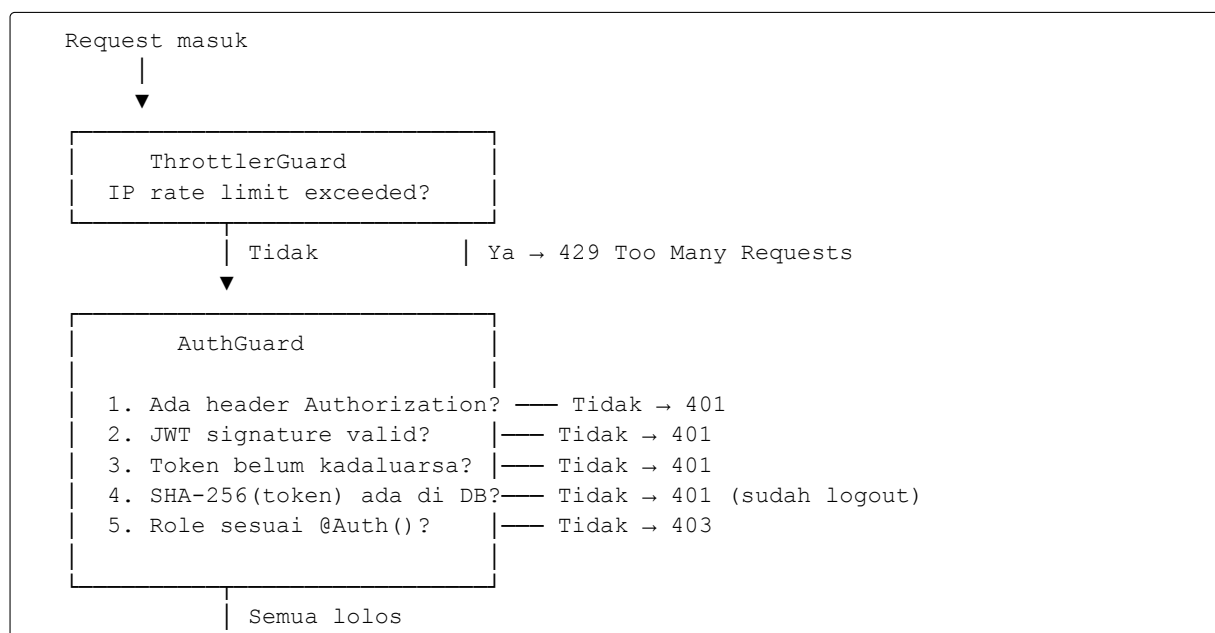
REQUEST TERAUTENTIKASI

1. Browser kirim request; cookie membawa JWT otomatis
2. ThrottlerGuard periksa rate limit per IP
3. AuthGuard:
 - a. Ekstrak JWT dari Authorization header
 - b. jwtService.verify(token) → payload { id, role, exp }
 - c. Cek exp - jika kadaluarsa → 401 Unauthorized
 - d. Hitung incomingHash = SHA-256(token)
 - e. Query DB: SELECT * FROM {role}s WHERE token = incomingHash
 - f. Jika tidak cocok (sudah logout) → 401 Unauthorized
 - g. @Auth() periksa role → jika salah → 403 Forbidden
4. Handler dieksekusi

LOGOUT

1. POST /auth/{role}/logout dengan JWT valid
2. Backend set kolom `token` = NULL di DB
3. Frontend hapus cookie: Set-Cookie: token=; Max-Age=0
4. Token lama tidak lagi valid (hash tidak ada di DB)

3.3. Diagram Verifikasi Token



▼
Handler dieksekusi

4. Protokol Komunikasi

4.1. Spesifikasi Kanal

Kanal	Protokol	Port	Persyaratan Keamanan
Browser → Frontend	HTTPS/TLS 1.2+	443	Sertifikat valid; HSTS direkomendasikan
Browser → Backend (via Frontend)	HTTPS/TLS 1.2+	443	Bearer JWT di Authorization: Bearer <token>
Frontend → Backend	HTTPS/TLS 1.2+	3000/443	Server-to-server; API route Next.js → NestJS
Backend → MySQL	TCP (TLS opsional)	3306	Koneksi hanya dari host lokal atau jaringan privat
Backend → OpenAI/OpenRouter	HTTPS/TLS 1.2+	443	API Key di header; payload teks saja
Backend → SMTP	SMTP/STARTTLS	587	Autentikasi SMTP; koneksi terenkripsi

Tabel 4: Spesifikasi kanal komunikasi

4.2. Header Keamanan HTTP

Header yang wajib dikonfigurasi pada server Next.js (`next.config.js`):

```
Content-Security-Policy:
  default-src 'self';
  script-src 'self' 'nonce-{NONCE}';
  connect-src 'self' https://api.backend.bacarita.id;

X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
Permissions-Policy: camera=self microphone=self
Strict-Transport-Security: max-age=63072000; includeSubDomains
```

`Permissions-Policy: camera=self microphone=self` diperlukan karena Bacarita menggunakan kamera (eye-tracking MediaPipe) dan mikrofon (ASR) — izin diberikan hanya ke origin sendiri.

5. Perubahan Struktur Database

Skema tambahan yang diperlukan untuk mengimplementasikan remediasi:

```
-- [AUTH-03] Hash token di semua tabel pengguna
-- token TEXT NULL → menyimpan SHA-256(JWT), bukan JWT asli
-- (kolom sudah ada, hanya perubahan cara pengisian)

-- [USER-03] Account lockout – tambahkan ke semua tabel pengguna
ALTER TABLE `teachers`
  ADD COLUMN `failedLoginAttempts` INT NOT NULL DEFAULT 0,
  ADD COLUMN `lockedUntil` DATETIME NULL;

ALTER TABLE `students`
  ADD COLUMN `failedLoginAttempts` INT NOT NULL DEFAULT 0,
  ADD COLUMN `lockedUntil` DATETIME NULL;

ALTER TABLE `parents`
  ADD COLUMN `failedLoginAttempts` INT NOT NULL DEFAULT 0,
  ADD COLUMN `lockedUntil` DATETIME NULL;

ALTER TABLE `admins`
  ADD COLUMN `failedLoginAttempts` INT NOT NULL DEFAULT 0,
  ADD COLUMN `lockedUntil` DATETIME NULL;

ALTER TABLE `curators`
  ADD COLUMN `failedLoginAttempts` INT NOT NULL DEFAULT 0,
  ADD COLUMN `lockedUntil` DATETIME NULL;

-- [AUTH-06] Audit log autentikasi (opsional namun direkomendasikan)
CREATE TABLE `auth_audit_logs` (
  `id` INT NOT NULL AUTO_INCREMENT,
  `userId` VARCHAR(255) NOT NULL,
  `role` ENUM('admin','teacher','student','parent','curator') NOT NULL,
  `event` ENUM('LOGIN_OK','LOGIN_FAIL','LOGOUT','LOCKED') NOT NULL,
  `ipAddress` VARCHAR(45) NULL,
  `userAgent` TEXT NULL,
  `createdAt` DATETIME(6) NOT NULL DEFAULT CURRENT_TIMESTAMP(6),
  PRIMARY KEY (`id`),
  INDEX `idx_userId` (`userId`),
  INDEX `idx_event_time` (`event`, `createdAt`)
) ENGINE=InnoDB;
```